Soluciones: Strings

Programación I, Semana 10

Ejercicios obligatorios

Ejercicio 1: Contar una letra

```
int contarLetra(std::string s, char c) {
    int n = s.size();
    int cuenta = 0;
    for (int i = 0; i < n; i++) {
        if (s[i] == c) {
            cuenta++;
        }
    }
    return cuenta;
}
```

Línea de `main`:

```
std::cout << contarLetra("banana", 'a');
```

Por qué: se revisa cada carácter y, si es igual al buscado, el contador sube en 1.
Errores frecuentes:

- `s[i] == 'c'`: compara con la letra c, no con el parámetro `c`. Sin comillas se usa la variable.
- `contarLetra("banana", "a")`: el segundo argumento es un `char`, con comillas simples.
- `cuenta = cuenta + s[i];`: eso es un acumulador. Aquí se necesita un contador.

Ejercicio 2: Seguimiento

`i`	`s[i]`	`¿s[i] == 't'?`	`pos`
inicio			−1
0	c	no	−1
1	o	no	−1
2	m	no	−1
3	p	no	−1
4	u	no	−1
5	t	sí	5

La función devuelve **5**.
¿Por qué no revisa `s[6]`? Cuando `i` pasa a valer 6, `pos` ya vale 5. La condición `pos == -1` es falsa, así que el ciclo termina. La búsqueda se detiene en cuanto encuentra la letra.

Ejercicio 3: ¿Qué imprime?

a) Imprime `Hola 4`. Se cambió solo el carácter 0; el tamaño sigue siendo 4.
b) Imprime `sol`. La función recibe una **copia** del string y cambia la copia. A diferencia de un arreglo, el string original no cambia.
c) Imprime `abcdab d`. El string `b` es `"ab" + "cd" + "ab"`, es decir `"abcdab"`. Sus índices son a(0) b(1) c(2) d(3) a(4) b(5), así que `b[3]` es `d`.

Ejercicio 4: Encontrar los errores

N.º	Error	Corrección
1	`int c;`: el contador no empieza en 0 y contiene basura.	`int c = 0;`
2	`i <= n`: en la última vuelta lee `s[n]`, fuera del string.	`i < n`
3	`s[i] == "e"`: compara un char con un string. No compila.	`s[i] == 'e'`
4	La función dice que devuelve `int`, pero no tiene `return`.	Agregar `return c;` antes de la `}` final.

Código corregido:

```
int contarVocales(std::string s) {
    int n = s.size();
    int c = 0;
    for (int i = 0; i < n; i++) {
        if (s[i] == 'a' || s[i] == 'e' || s[i] == 'i' ||
            s[i] == 'o' || s[i] == 'u') {
            c++;
        }
    }
    return c;
}
```

Las vocales de `"murcielago"` son u, i, e, a, o: imprime **5**.

Ejercicios intermedios

Ejercicio 5: Palíndromo

```
bool esPalindromo(std::string s) {
    int n = s.size();
    bool es = true;
    for (int i = 0; i < n / 2; i++) {
        if (s[i] != s[n - 1 - i]) {
            es = false;
        }
    }
    return es;
}
```

Por qué: se supone que sí es palíndromo y se busca una pareja que no coincida. Basta con recorrer hasta la mitad (`n / 2`), porque cada vuelta revisa un carácter del principio y uno del final.
Con `"reconocer"` todas las parejas coinciden (r-r, e-e, c-c, o-o): devuelve **true**. Con `"programa"` la primera pareja ya falla (p-a): devuelve **false**.
También es correcto: recorrer el string completo con `i < n`, que revisa cada pareja dos veces. O construir el string invertido, como en la función `invertir` de la clase, y comparar con `==`.
Errores frecuentes:

- `s[n - i]`: con `i = 0` lee `s[n]`, fuera del string. La pareja correcta es `s[n - 1 - i]`.
- Poner `es = true` dentro del ciclo cuando una pareja coincide: borra un `false` anterior.

Reto

Ejercicio 6: Lista de nombres

a)

```
int contarInicial(std::string nombres[], int n, char letra) {
    int c = 0;
    for (int i = 0; i < n; i++) {
        if (nombres[i][0] == letra) {
            c++;
        }
    }
    return c;
}
```

Empiezan con A: Ana, Andres y Alberto. Devuelve **3**.
b)

```
int buscarNombre(std::string nombres[], int n, std::string x) {
    int pos = -1;
    for (int i = 0; i < n && pos == -1; i++) {
        if (nombres[i] == x) {
            pos = i;
        }
    }
    return pos;
}
```

`"Maria"` está en la posición **3**; `"Pedro"` no está y devuelve **−1**.
Detalle: aquí `n` es un parámetro, porque se recorre un **arreglo** de strings. El arreglo no conoce su tamaño, aunque sus elementos sí. Además, `nombres[i] == x` compara strings completos con `==`, algo que con arreglos no se puede hacer.